

# Tutorial: A Lightweight Web Application for Software Vulnerability Demonstration

David Lee<sup>1</sup>, Brandon Steed<sup>1</sup>, Yi Liu<sup>2</sup>, Onyeka Ezenwoye<sup>1</sup>

<sup>1</sup> Augusta University, Augusta, GA, USA, {davlee, bsteed1, oezenwoye}@augusta.edu

<sup>2</sup> University of Massachusetts Dartmouth, Dartmouth, MA, USA, yliu11@umassd.edu

**Abstract**—In cybersecurity education, it is critical to introduce students to security concepts and keep them aware of common software security weaknesses. However, the effectiveness of delivering such knowledge is complicated by the lack of practical security content and a case study that embeds contemporary security vulnerabilities for education.

In this tutorial, we propose to introduce the participants to common web application vulnerabilities using a novel lightweight application case study that demonstrates software security weaknesses in a practical manner. With this tutorial, we will facilitate discussion about ways to expose students to security weaknesses, how to mitigate software vulnerabilities through secure software design and coding practices, as well as share ideas on how to improve the case study application.

This three-part tutorial will first introduce the National Vulnerability Database. In addition, we will discuss the Common Weakness Enumeration and the relationship between vulnerabilities and weaknesses. We will then illustrate how the database is used to derive the common web application weaknesses. The second part of the tutorial will demonstrate the lightweight web application as a case study to illustrate the most common web application weaknesses. The participants will be guided on how to download and use the web application. This will include practical exercises of how to activate the built-in vulnerabilities that expose the common security weaknesses. Lastly, we will facilitate a discussion on the efficacy of the case study as a means for practical software vulnerability demonstration for education with a view on ways to enhance the case study.

**Keywords:** Web Application, Weakness, Vulnerability

## I. INTRODUCTION

To help address the prevalence of software vulnerabilities, the National Institute of Standards and Technology maintains a database of vulnerabilities that exist in everyday software [2]. This initiative has cataloged a list of vulnerabilities over the years. Each vulnerability is mapped to a weakness enumeration. The weakness enumeration provides a means of categorizing each vulnerability to a type of weakness (E.g., Improper Input Validation) [1]. The authors have classified the impacted software products in the database into seven software types (Web Application, Utility, Server, Operating System, Web Browser, Framework, and Middleware). The vulnerabilities for the Web Application type are reviewed to determine the most common weaknesses (Table I) over the past decade (2011 - 2020)

As web applications have become ubiquitous, there is a need to address the proliferation of the associated software vulnerabilities. With this tutorial, we intend to shed light on the common Web application weaknesses, in a practical

way. This is to better inform stakeholders in the push to improve the engineering of software. The database catalogs vulnerabilities in a list known as Common Vulnerabilities and Exposures (CVE). Each entry in the list contains several attributes including the name of the software product, its vendor, and a description of the vulnerability. Each CVE entry is also mapped to an item in the Common Weakness Enumeration (CWE) [1]. From here on, we refer to CVE entries as vulnerability and CWE items as weakness. Ten of the most common weaknesses are built into a case study web application for demonstration [3] We define the most common as weaknesses that occur at a rate of 1% or higher of all web application vulnerabilities for the past decade.

We propose a tutorial session on the demonstration of these weaknesses. The tutorial will include hands-on components and audience interactions. The three parts of the tutorial are presented below:

TABLE I  
MOST COMMON WEB APPLICATION WEAKNESSES

| Name                              | Weight |
|-----------------------------------|--------|
| Cross-Site Scripting (XSS)        | 27.6%  |
| SQL Injection                     | 9.9%   |
| Exposure of Sensitive Information | 7.0%   |
| Cross-Site Request Forgery (CSRF) | 5.8%   |
| Improper Access Control           | 5.2%   |
| Improper Input Validation         | 4.5%   |
| Path Traversal                    | 3.3%   |
| Buffer Overflow                   | 2.3%   |
| Unrestricted Upload of File       | 2.0%   |
| Code Injection                    | 1.8%   |
| Improper Authentication           | 1.0%   |

### A. Introduction to Common Software Weaknesses

In this section, we will illustrate the process through which we have analyzed the vulnerability data in the database in order to derive the common web application weaknesses. This will familiarize participants with the database and examples of vulnerabilities.

### B. Demonstration of Insecure Web Application

To illustrate the common weaknesses, the case study (a course registration system) will be used to walk the participants through examples of the following weaknesses:

- 1) *Cross-site Scripting (XSS)*: A web browser's ability to run embedded code can be exploited by inserting

malicious web scripts (e.g., JavaScript) into system data. The script could be inserted several vectors (E.g., form data or file). The malicious code could redirect the user or user's data to another domain [4]. Mitigating XSS includes avoiding the use of user-supplied input in application logic, and validating both application input and output.

- 2) *SQL Injection*: This is a type of attack that occurs when an attacker can insert SQL statements into application data. The data is designed to exploit improper validation, sanitization, and poorly constructed queries to gain access to the database [4]. Mitigating SQL injection includes data validation, sanitization, and avoiding the use of user-supplied input in SQL queries. Also, making use of stored procedures rather than dynamic queries, and queries that are resilient to SQL injection.
- 3) *Exposure of Sensitive Information to an Unauthorized Actor*: An application could reveal important information about its implementation, state, or other confidential information (E.g., IP addresses or database connection). These disclosures could occur through descriptive error messages, insecure cookies, cache, URL parameters, and network traffic. An attacker could utilize this information to refine an attack. Mitigation includes using generic error messages, effective access control, strong data encryption, and limiting client-side information exposure.
- 4) *Cross-Site Request Forgery (CSRF)*: This weakness exists when an application is unable to correctly determine if a client request is unintentionally sent by the user. This improper verification of request could be exploited to trick a user into clicking a link to submit a malicious request. This request leverages the context of the user's valid session [5]. Mitigation includes using unique request tokens and inserting tokens into hidden form fields.
- 5) *Improper Access Control*: This weakness includes failure in correctly determining if the actor is authorized to access some functionality or information. This could result from the improper management of session identifiers as well as lack of consistent enforcement of authentication and authorization across different layers of the architecture. Mitigation includes the use of appropriate access control patterns (E.g., role-based access control) as well as strong policies for managing user credentials.
- 6) *Improper Input Validation*: Not properly validating and sanitizing consumed data can cause an application to enter an erroneous state, which could lead to other vulnerabilities [7].
- 7) *Path Traversal*: Applications may use input data to generate a path to the file system. This behavior could be exploited to gain unauthorized access to the file system and adversely impact the application (E.g., availability and confidentiality) [6]. For instance, a malicious URL could be crafted by replacing portions of a legitimate URL with a dot-dot-slash sequence (E.g., ../..) to gain access to some directory. Mitigation includes not allowing user-controlled data as input to the file system, data

sanitization, and effective access control measures.

- 8) *Unrestricted Upload of File*: This weakness relates to allowing an attacker to specify a malicious file that is consumed by the application. The file is then either automatically processed by the application or directly accessed by the attacker. Mitigation requires that both a file's payload and type are adequately validated. Whitelists or blacklists could be used to filter allowable types. Also, access control measures should be used to prevent unauthorized uploads.
- 9) *Code Injection*: This includes vulnerabilities that allow an attacker to insert code snippets into a system and thereby alter its behavior. Mitigation includes validation and sanitization of data. Also, access control measures to prevent access to features and components that are susceptible to code injection.
- 10) *Improper Authentication*: This involves the insufficient verification of the identity of a user, which results in unauthorized access. This weakness is a *child-of* Improper Access Control [1].

### C. Discussion of Case Study

A crucial aspect of this tutorial is to get some feedback from the participants on both the process for deriving the most common web application weaknesses and the efficacy of the case study web application in imparting cybersecurity knowledge in a practical manner. The authors will facilitate a discussion on ways to improve both aspects.

## II. PREREQUISITES AND TARGET AUDIENCE

This tutorial requires no programming experience. Participants will need to have access to a personal computer that is running a recent version of Microsoft Windows (preferably Windows 10). The application (67MB) will need to be downloaded but no installation is required. The target audience for this tutorial includes students, faculty, security professionals, and any persons who are interested in learning about common software security vulnerabilities. It is the intention of the authors that this tutorial will facilitate discussion about ways to expose students to security weaknesses in a practical way, how to mitigate these vulnerabilities through secure software design and coding practices, as well as share ideas on how to improve the case study application.

## REFERENCES

- [1] Common weakness enumeration. <https://nvd.nist.gov/vuln/categories>, Accessed 3/21.
- [2] National Vulnerability Database. <https://nvd.nist.gov/>, Accessed 3/21.
- [3] SecureED. <https://github.com/onniggit/SecureEd>, Accessed 9/21.
- [4] M. Andrews and J. A. Whittaker. *How to Break Web Software: Functional and Security Testing of Web Applications and Web Services*. Addison-Wesley Professional, 2006.
- [5] B. Chess and J. West. *Secure Programming with Static Analysis*. Addison-Wesley Professional, first edition, 2007.
- [6] M. Dowd, J. McDonald, and J. Schuh. *The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities*. Addison-Wesley Professional, 2006.
- [7] K. R. van Wyk, M. G. Graff, D. S. Peters, and D. L. Burley. *Enterprise Software Security: A Confluence of Disciplines*. Addison-Wesley Professional, 1st edition, 2014.